Experiment No. 5

Aim :

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning

Theory :

1. Dataset Source

Dataset Name: **Pima Indians Diabetes Database**
Source Platform: Kaggle

Dataset Link :
https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database?resource=download

2. Dataset Description

The Pima Indians Diabetes dataset consists of medical attributes used to determine whether a patient has diabetes. The dataset contains several physiological measurements and medical test results.

Features in the Dataset:

- **Pregnancies** – Number of times the patient has been pregnant
- **Glucose** – Plasma glucose concentration
- **BloodPressure** – Diastolic blood pressure (mm Hg)
- **SkinThickness** – Triceps skin fold thickness (mm)
- **Insulin** – 2-Hour serum insulin (mu U/ml)
- **BMI** – Body Mass Index
- **DiabetesPedigreeFunction** – Diabetes pedigree score (genetic influence)
- **Age** – Age of the patient

Target Variable:

Outcome

- 0 → No Diabetes
- 1 → Diabetes

Dataset Characteristics:

- Total records: **768**
- Number of features: **8**
- Binary classification problem
- Contains only numeric attributes
- Suitable for machine learning classification algorithms such as SVM.

3. Mathematical Formulation of the Algorithm

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. In classification problems, SVM aims to find the **optimal hyperplane** that separates data points belonging to different classes.

For a linear SVM, the decision boundary can be represented as:

$$\boxed{w \cdot x + b = 0}$$

The objective of SVM is to **maximize the margin** between the two classes while minimizing classification errors.

The optimization problem can be expressed as:

$$\boxed{\min \frac{1}{2}||w||^2}$$

subject to:

$$\boxed{y_i(w \cdot x_i + b) \geq 1}$$

For non-linear data, SVM uses **kernel functions** such as:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF) Kernel
- Sigmoid Kernel

These kernels transform the input data into a higher dimensional space to make it linearly separable.

4. Algorithm Limitations

Although SVM is a powerful classification algorithm, it has some limitations:

- **High Computational Cost:** Training can be slow for very large datasets.
- **Sensitive to Parameter Selection:** Performance depends heavily on hyperparameters such as C and gamma.
- **Not Suitable for Very Large Datasets:** Memory usage increases with large training data.
- **Kernel Selection Complexity:** Choosing the correct kernel function can be difficult.
- **Less Interpretable:** Compared to decision trees or logistic regression, SVM models are harder to interpret.

5. Methodology / Workflow

Step 1: Data Collection

The diabetes dataset was downloaded from Kaggle and loaded into a Pandas DataFrame.

Step 2: Data Exploration

The dataset structure, feature types, and summary statistics were examined using functions such as head(), info(), and describe().

Step 3: Data Visualization

Several visualization techniques were used to understand data patterns:

- Outcome distribution plot
- Feature distribution histograms
- Correlation heatmap

These visualizations helped understand relationships between medical features and diabetes outcome.

Step 4: Data Preparation

The dataset was divided into:

- **Features (X)**
- **Target variable (y)**

Step 5: Train-Test Split

The dataset was divided into training and testing sets using an **80:20 ratio**.

Step 6: Feature Scaling

StandardScaler was applied to normalize feature values because SVM is sensitive to feature magnitudes.

Step 7: Model Training

An SVM classifier was trained using the training dataset.

Step 8: Model Evaluation

The model performance was evaluated using:

- Accuracy score
- Confusion matrix
- Classification report

6. Performance Analysis

The performance of the SVM model was evaluated using standard classification metrics.

- **Accuracy Score:** Indicates the percentage of correctly classified instances.
- **Confusion Matrix:** Shows correct and incorrect predictions made by the model.
- **Precision:** Measures how many predicted positives are actually correct.
- **Recall:** Measures how many actual positives were correctly identified.
- **F1-score:** Harmonic mean of precision and recall.

```python
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7337662337662337

Confusion Matrix:
[[82 17]
 [24 31]]

Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.83      0.80        99
           1       0.65      0.56      0.60        55

    accuracy                           0.73       154
   macro avg       0.71      0.70      0.70       154
weighted avg       0.73      0.73      0.73       154
```

7. Hyperparameter Tuning

Hyperparameter tuning was performed using **GridSearchCV** to find the optimal parameters for the SVM model.

The following parameters were tuned:

- **C (Regularization Parameter):** Controls the trade-off between maximizing margin and minimizing classification error.
- **Gamma:** Determines the influence of individual training samples.
- **Kernel:** Defines the type of decision boundary used by the SVM model.

Different combinations of parameters were tested using cross-validation. The best performing parameters were selected based on the highest accuracy score.

Output :

[2]
4s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_repo
```

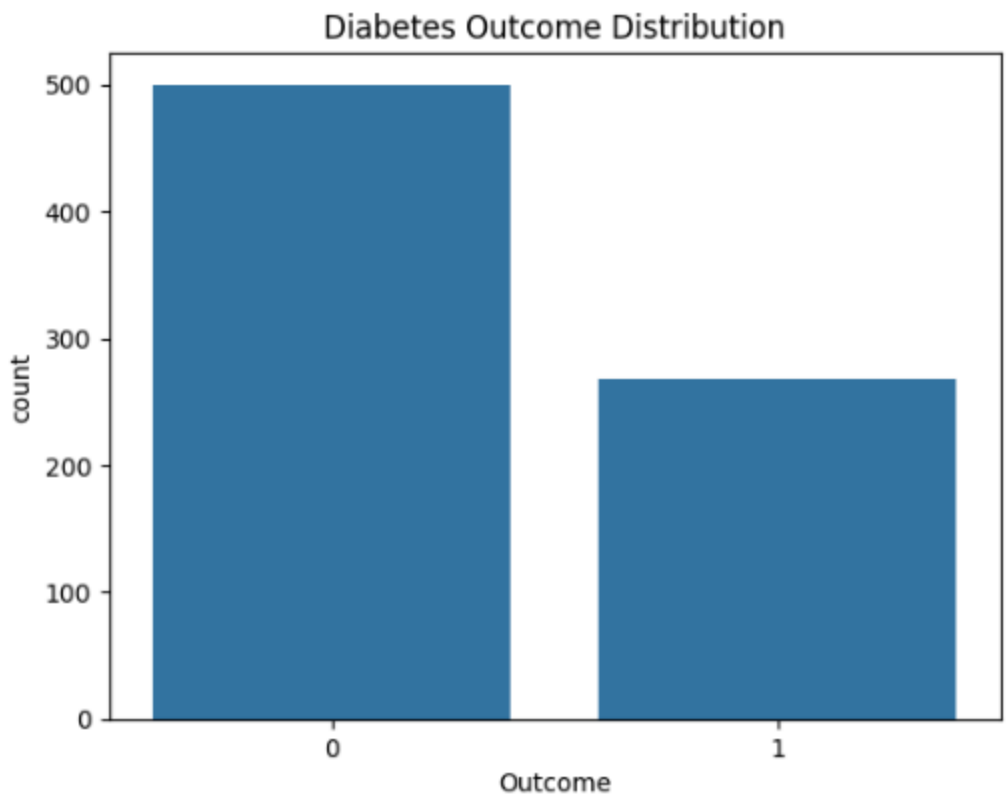
```python
df = pd.read_csv("diabetes.csv")
print(df.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

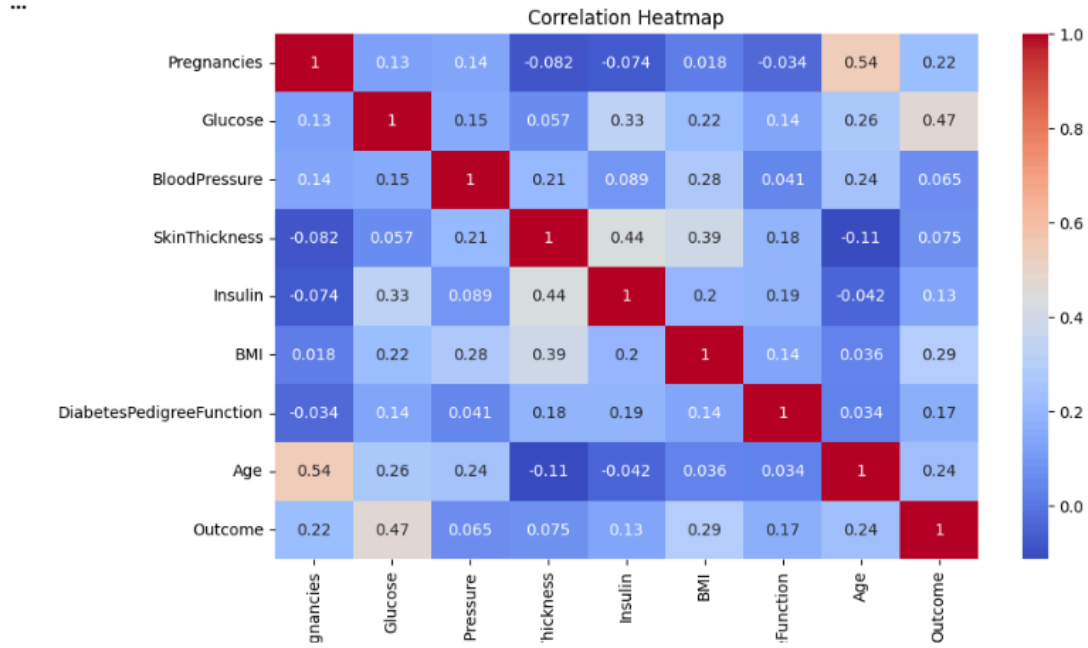
```python
print(df.info())
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
```

```python
sns.countplot(x='Outcome', data=df)
plt.title("Diabetes Outcome Distribution")
plt.show()
```



```python
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```




```python
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
```

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```python
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```python
svm_model = SVC()

svm_model.fit(X_train, y_train)
```

```
SVC
SVC()
```

```python
y_pred = svm_model.predict(X_test)
```

```python
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.7337662337662337

Confusion Matrix:
[[82 17]
 [24 31]]

Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.83      0.80        99
           1       0.65      0.56      0.60        55

    accuracy                           0.73       154
   macro avg       0.71      0.70      0.70       154
weighted avg       0.73      0.73      0.73       154
```

```python
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [1, 0.1, 0.01, 0.001],
    'kernel': ['rbf', 'linear']
}

grid = GridSearchCV(SVC(), param_grid, refit=True, verbose=2)

grid.fit(X_train, y_train)
```

```
Fitting 5 folds for each of 32 candidates, totalling 160 fits
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.1s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.1s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.1s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.1s
[CV] END .........................C=0.1, gamma=1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.1, kernel=rbf; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
[CV] END ....................C=0.1, gamma=0.1, kernel=linear; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.01, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.01, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.01, kernel=rbf; total time=   0.0s
[CV] END ......................C=0.1, gamma=0.01, kernel=rbf; total time=   0.0s
```

```
[CV] END ...................C=100, gamma=0.01, kernel=linear; total time=   0.2s
[CV] END ...................C=100, gamma=0.01, kernel=linear; total time=   0.2s
[CV] END ...................C=100, gamma=0.01, kernel=linear; total time=   0.2s
[CV] END ...................C=100, gamma=0.01, kernel=linear; total time=   0.2s
[CV] END .....................C=100, gamma=0.001, kernel=rbf; total time=   0.0s
[CV] END .....................C=100, gamma=0.001, kernel=rbf; total time=   0.0s
[CV] END .....................C=100, gamma=0.001, kernel=rbf; total time=   0.0s
[CV] END .....................C=100, gamma=0.001, kernel=rbf; total time=   0.0s
[CV] END .....................C=100, gamma=0.001, kernel=rbf; total time=   0.0s
[CV] END ..................C=100, gamma=0.001, kernel=linear; total time=   0.3s
[CV] END ..................C=100, gamma=0.001, kernel=linear; total time=   0.2s
[CV] END ..................C=100, gamma=0.001, kernel=linear; total time=   0.2s
[CV] END ..................C=100, gamma=0.001, kernel=linear; total time=   0.3s
[CV] END ..................C=100, gamma=0.001, kernel=linear; total time=   0.4s
```

GridSearchCV

best_estimator_:
SVC

SVC

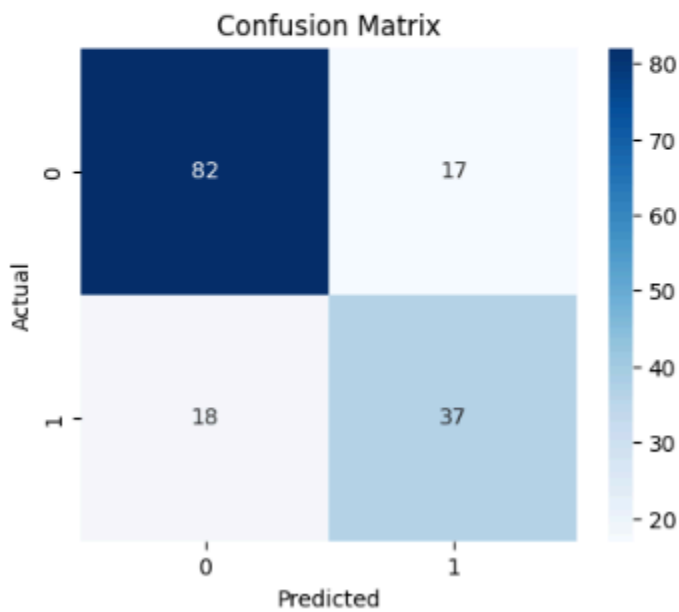
```
print("Best Parameters:", grid.best_params_)
```

```
Best Parameters: {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}
```

```
best_model = grid.best_estimator_

y_pred_best = best_model.predict(X_test)

print("Final Accuracy:", accuracy_score(y_test, y_pred_best))
```

```
Final Accuracy: 0.7727272727272727
```

```
cm = confusion_matrix(y_test, y_pred_best)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Conclusion :

In this experiment, the Support Vector Machine (SVM) algorithm was successfully implemented to classify diabetes outcomes using medical attributes. After preprocessing the dataset and applying feature scaling, the model was trained and evaluated using various performance metrics. Hyperparameter tuning using GridSearchCV improved the classification performance. The experiment demonstrated the effectiveness of SVM in solving binary classification problems in healthcare datasets.